

**République Tunisienne**

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

---

**École Nationale d'Ingénieurs de Gabès – ENIG**

Département Génie Électrique et Automatique

---

## **Rapport de Projet**

**Génie Électrique et Automatique – GEA**

### **Système de Reconnaissance Faciale sur Raspberry Pi**

*Détection et identification en temps réel  
via caméra USB et écran intégré*

**Réalisé par :**

MAHMOUDI LASSAAD

Classe : GEA 2

**Encadrant :**

M.MOHAMED NAQUI

Maître

**Année universitaire 2025 – 2026**



# Remerciements

Mes remerciements s'adressent en premier lieu à **M. MOHAMED NAOUI** pour sa disponibilité, ses conseils avisés et l'intérêt constant qu'il a porté à l'avancement de ce travail.

# Résumé

Ce projet porte sur la conception et la réalisation d'un système de **reconnaissance faciale embarqué** sur une carte **Raspberry Pi 4**. Le système utilise une **caméra USB** pour capturer les visages en temps réel et un **écran** pour afficher les résultats d'identification directement sur le dispositif, sans nécessiter de connexion à un ordinateur externe.

L'algorithme de reconnaissance repose sur le modèle **LBPH** (Local Binary Pattern Histogram) intégré dans la bibliothèque OpenCV, complété par un détecteur de visages basé sur les **cascades de Haar**. Trois scripts Python indépendants assurent respectivement la collecte du jeu de données, l'entraînement du modèle et la reconnaissance en temps réel.

Les tests menés sur trois sujets dans des conditions d'éclairage variées ont montré une précision de **94 %** avec un débit de traitement de **8–9 images par seconde**, garantissant une interaction fluide et réactive.

**Mots-clés :** Raspberry Pi, reconnaissance faciale, LBPH, OpenCV, caméra USB, vision par ordinateur, systèmes embarqués.

# Abstract

This project focuses on the design and implementation of an **embedded facial recognition system** based on a **Raspberry Pi 4** single-board computer. The system uses a **USB camera** to capture faces in real time and a **display screen** to show identification results directly on the device, with no need for an external computer.

The recognition algorithm relies on the **LBPH** (Local Binary Pattern Histogram) model integrated within the **OpenCV** library, combined with a face detector based on **Haar cascades**. Three independent Python scripts handle dataset collection, model training, and real-time recognition, respectively.

Tests conducted on three subjects under varying lighting conditions achieved a recognition accuracy of **94 %** at a processing rate of **8–9 frames per second**, ensuring smooth and responsive interaction.

**Keywords :** Raspberry Pi, facial recognition, LBPH, OpenCV, USB camera, computer vision, embedded systems.

# Table des matières

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Remerciements</b>                                                         | <b>2</b>  |
| <b>Résumé</b>                                                                | <b>3</b>  |
| <b>Abstract</b>                                                              | <b>4</b>  |
| <b>Liste des figures</b>                                                     | <b>7</b>  |
| <b>Liste des tableaux</b>                                                    | <b>8</b>  |
| <b>Liste des abréviations</b>                                                | <b>9</b>  |
| <b>Introduction Générale</b>                                                 | <b>10</b> |
| <b>1 Généralités sur la reconnaissance faciale et les systèmes embarqués</b> | <b>11</b> |
| 1.1 La reconnaissance faciale : définition et enjeux .....                   | 11        |
| 1.2 Évolution historique .....                                               | 11        |
| 1.2.1 Années 1960 – 1990 : approches géométriques .....                      | 11        |
| 1.2.2 Années 1990 : les eigenfaces .....                                     | 11        |
| 1.2.3 Années 2000 : les approches locales (LBP) .....                        | 12        |
| 1.2.4 Depuis 2012 : le deep learning .....                                   | 12        |
| 1.3 État de l’art .....                                                      | 12        |
| <b>2 Analyse et conception du système</b>                                    | <b>13</b> |
| 2.1 Cahier des charges .....                                                 | 13        |
| 2.2 Architecture globale .....                                               | 13        |
| 2.3 Flux de traitement d’une image .....                                     | 14        |
| 2.4 Choix de conception .....                                                | 14        |
| 2.4.1 Choix de l’algorithme de reconnaissance .....                          | 14        |
| 2.4.2 Choix du détecteur de visages .....                                    | 14        |
| <b>3 Matériels et outils</b>                                                 | <b>15</b> |
| 3.1 Composants matériels .....                                               | 15        |
| 3.1.1 Vue d’ensemble .....                                                   | 15        |
| 3.1.2 La carte Raspberry Pi 4 Model B .....                                  | 17        |
| 3.1.3 La caméra USB .....                                                    | 17        |

|          |                                                                                                                                                        |           |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 3.1.4    | L'écran .....                                                                                                                                          | 17        |
| 3.1.5    | Schéma de câblage du système .....                                                                                                                     | 17        |
| 3.2      | Environnement logiciel .....                                                                                                                           | 18        |
| 3.2.1    | OpenCV .....                                                                                                                                           | 18        |
| 3.3      | Algorithme LBPH.....                                                                                                                                   | 18        |
| 3.3.1    | Principe général.....                                                                                                                                  | 18        |
| 3.3.2    | Formule mathématique du LBP.....                                                                                                                       | 19        |
| 3.3.3    | Distance du $\chi^2$ pour la comparaison .....                                                                                                         | 19        |
| 3.3.4    | Paramètres retenus .....                                                                                                                               | 19        |
| 3.4      | Architecture logicielle.....                                                                                                                           | 19        |
| 3.4.1    | Script 1 : collecte du dataset (face_dataset.py).....                                                                                                  | 19        |
| 3.4.2    | Script 2 : entraînement du modèle (face_train.py) .....                                                                                                | 20        |
| 3.4.3    | Script 3 : reconnaissance en temps réel (face_recognition_app.py) .....                                                                                | 21        |
| <b>4</b> | <b>Réalisation et résultats</b>                                                                                                                        | <b>23</b> |
| 4.1      | Mise en œuvre du système.....                                                                                                                          | 23        |
| 4.1.1    | Installation de l'environnement.....                                                                                                                   | 23        |
| 4.1.2    | Procédure d'utilisation.....                                                                                                                           | 23        |
| 4.2      | Capture d'écran de la reconnaissance en temps réel.....                                                                                                | 23        |
| 4.3      | Conditions de test .....                                                                                                                               | 24        |
| 4.4      | Résultats de performance.....                                                                                                                          | 24        |
| 4.5      | Analyse des résultats .....                                                                                                                            | 24        |
| 4.5.1    | Points forts.....                                                                                                                                      | 24        |
| 4.5.2    | Limites observées .....                                                                                                                                | 25        |
| 4.6      | Perspectives d'amélioration.....                                                                                                                       | 25        |
|          | <b>Conclusion Générale</b>                                                                                                                             | <b>26</b> |
|          | <b>Bibliographie</b>                                                                                                                                   | <b>27</b> |
| <b>A</b> | <b>Code source complet</b>                                                                                                                             | <b>28</b> |
| A.1      | Script complet : collecte du dataset .....                                                                                                             | 28        |
| A.2      | Script complet : entraînement.....                                                                                                                     | 29        |
| A.3      | Script complet : reconnaissance en temps réel.....                                                                                                     | 30        |
| 3.1      | Carte Raspberry Pi 4 Model B et ses principaux connecteurs.....                                                                                        | 17        |
| 3.2      | Schéma de câblage du système : Raspberry Pi 4, caméra USB et écran HDMI .....                                                                          | 17        |
| 4.1      | Capture d'écran du système en fonctionnement : détection et reconnaissance d'un visage avec affichage du nom et du score de confiance sur l'écran..... | 24        |

# Liste des tableaux

|     |                                                                       |    |
|-----|-----------------------------------------------------------------------|----|
| 1.1 | Comparaison des principales approches de reconnaissance faciale ..... | 12 |
| 2.1 | Flux de traitement d'une frame .....                                  | 14 |
| 3.1 | Composants matériels du système .....                                 | 16 |
| 3.2 | Bibliothèques et modules Python utilisés.....                         | 18 |
| 3.3 | Paramètres du modèle LBPH.....                                        | 19 |
| 4.1 | Conditions expérimentales des tests.....                              | 24 |
| 4.2 | Résultats de performance du système .....                             | 24 |



# Liste des abréviations

| Abréviation | Signification                                                |
|-------------|--------------------------------------------------------------|
| ACP         | Analyse en Composantes Principales                           |
| ARM         | Advanced RISC Machine                                        |
| CNN         | Convolutional Neural Network (Réseau de neurones convolutif) |
| CPU         | Central Processing Unit                                      |
| FPS         | Frames Per Second (Images par seconde)                       |
| GPIO        | General Purpose Input/Output                                 |
| GPU         | Graphics Processing Unit                                     |
| LBP         | Local Binary Pattern (Motif binaire local)                   |
| LBPH        | Local Binary Pattern Histogram                               |
| OS          | Operating System (Système d'exploitation)                    |
| RAM         | Random Access Memory                                         |
| ROI         | Region of Interest (Région d'intérêt)                        |
| RPi         | Raspberry Pi                                                 |
| USB         | Universal Serial Bus                                         |

# Introduction Générale

## Contexte

Avec l'essor des systèmes embarqués et la démocratisation des nano-ordinateurs, il est aujourd'hui possible de déployer des applications d'intelligence artificielle directement sur des dispositifs à faible coût. La reconnaissance faciale, autrefois réservée aux infrastructures matérielles puissantes, peut désormais être exécutée localement sur une carte Raspberry Pi 4.

## Problématique

Comment concevoir un système de reconnaissance faciale fiable, autonome et peu coûteux, capable de fonctionner en temps réel avec comme seuls périphériques une **caméra USB** et un **écran**, sans recours à un serveur distant ni à un accélérateur matériel dédié ?

## Objectifs

Les objectifs de ce projet sont les suivants :

- Développer un système de détection et d'identification de visages fonctionnant en temps réel.
- Utiliser uniquement des composants standards et accessibles : Raspberry Pi 4, caméra USB et écran.
- Implémenter l'algorithme LBPH via OpenCV en Python.
- Évaluer les performances du système en termes de précision et de débit d'images.

## Organisation du rapport

Ce rapport est organisé en quatre chapitres :

1. **Chapitre 1** : généralités sur la reconnaissance faciale et les systèmes embarqués.
2. **Chapitre 2** : analyse et conception du système.
3. **Chapitre 3** : matériels et outils logiciels utilisés.
4. **Chapitre 4** : réalisation, tests et résultats.

# Chapitre 1

## Généralités sur la reconnaissance faciale et les systèmes embarqués

### Introduction

Ce chapitre présente le contexte général de la reconnaissance faciale, son évolution historique ainsi que les principales approches algorithmiques existantes. Il positionne notre projet par rapport à l'état de l'art et justifie les choix technologiques adoptés.

### 1.1 La reconnaissance faciale : définition et enjeux

La reconnaissance faciale est un processus biométrique permettant d'identifier ou de vérifier l'identité d'une personne à partir de son visage. Elle intervient dans de nombreux domaines : sécurité, contrôle d'accès, interaction homme-machine et domotique.

Contrairement aux systèmes basés sur des badges ou des codes PIN, la reconnaissance faciale est **non intrusive** : l'utilisateur n'a pas à effectuer de geste particulier pour être identifié, ce qui améliore considérablement l'ergonomie du système.

### 1.2 Évolution historique

#### 1.2.1 Années 1960 – 1990 : approches géométriques

Les premières tentatives d'automatisation (Bledsoe, Chan et Bisson, 1966) reposaient sur des mesures géométriques manuelles : distance inter-oculaire, largeur du nez, forme du menton. Ces approches nécessitaient une intervention humaine pour annoter les points caractéristiques.

#### 1.2.2 Années 1990 : les eigenfaces

Turk et Pentland (1991) ont introduit la méthode des *eigenfaces*, basée sur l'Analyse en Composantes Principales (ACP). Cette technique réduit la dimensionnalité des images et représente chaque visage dans un sous-espace propre, permettant une comparaison rapide et automatique.

### 1.2.3 Années 2000 : les approches locales (LBP)

Ojala *et al.* (2002) ont proposé le descripteur **Local Binary Pattern (LBP)**, qui encode la texture locale de l'image en comparant chaque pixel à ses voisins. L'extension LBPH améliore significativement la robustesse aux variations d'éclairage.

### 1.2.4 Depuis 2012 : le deep learning

L'avènement des réseaux de neurones convolutifs (CNN) a révolutionné le domaine. Des architectures comme *FaceNet* (Google) ou *ArcFace* atteignent des précisions supérieures à 99 % sur les benchmarks de référence. Toutefois, leur déploiement sur des systèmes embarqués à ressources limitées reste un défi majeur.

## 1.3 État de l'art

TABLE 1.1 – Comparaison des principales approches de reconnaissance faciale

| Approche         | Avantages                    | Inconvénients               | Précision |
|------------------|------------------------------|-----------------------------|-----------|
| Eigenfaces (ACP) | Simple, rapide               | Sensible à l'éclairage      | ~80 %     |
| LBPH             | Robuste à l'éclairage, léger | Sensible aux poses extrêmes | ~94 %     |
| FaceNet (CNN)    | Très précis, généraliste     | Lourd, GPU requis           | >99 %     |
| ArcFace (CNN)    | Meilleure marge de décision  | Ressources importantes      | >99 %     |

Notre projet retient l'approche **LBPH** comme compromis optimal entre précision et contraintes matérielles du Raspberry Pi 4.

#### Conclusion

Ce chapitre a rappelé l'évolution des techniques de reconnaissance faciale, des approches géométriques aux méthodes profondes. L'algorithme LBPH a été identifié comme le plus adapté à notre contexte embarqué. Le chapitre suivant présentera l'analyse et la conception détaillée du système.

# Chapitre 2

## Analyse et conception du système

### Introduction

Ce chapitre décrit le cahier des charges du système, son architecture globale, le flux de traitement des images et les justifications des choix de conception retenus.

### 2.1 Cahier des charges

Le système doit répondre aux exigences suivantes :

- **Matériel minimal** : Raspberry Pi 4, caméra USB, écran HDMI. Aucun périphérique réseau, aucun serveur distant, aucun accélérateur matériel supplémentaire.
- **Autonomie** : fonctionnement entièrement local.
- **Temps réel** : débit minimal de 5 images par seconde.
- **Extensibilité** : ajout de nouveaux utilisateurs sans réentraîner l'intégralité du modèle.
- **Interface** : affichage du nom et du score de confiance sur l'écran.

### 2.2 Architecture globale

Le système repose sur trois blocs fonctionnels :

1. **Bloc d'acquisition** : la caméra USB capture le flux vidéo et le transmet au processeur via le bus USB.
2. **Bloc de traitement** : le processeur ARM Cortex-A72 exécute les algorithmes de détection (Haar cascade) et de reconnaissance (LBPH).
3. **Bloc d'affichage** : les résultats sont affichés sur l'écran connecté en HDMI (nom, score, rectangle de détection).

## 2.3 Flux de traitement d'une image

TABLE 2.1 – Flux de traitement d'une frame

| Étape | Opération   | Description                                           |
|-------|-------------|-------------------------------------------------------|
| 1     | Capture     | Lecture d'une frame depuis la caméra USB              |
| 2     | Miroir      | Inversion horizontale pour un affichage naturel       |
| 3     | Conversion  | Passage en niveaux de gris                            |
| 4     | Égalisation | Égalisation d'histogramme pour améliorer le contraste |
| 5     | Détection   | Haar cascade → localisation du visage                 |
| 6     | Extraction  | Découpe et redimensionnement de la ROI (200×200 px)   |
| 7     | Prédiction  | LBPH → label + score de confiance                     |
| 8     | Décision    | Seuil 70 : reconnu ou « Inconnu »                     |
| 9     | Affichage   | Rectangle + nom affiché sur l'écran                   |

## 2.4 Choix de conception

### 2.4.1 Choix de l'algorithme de reconnaissance

L'algorithme LBPH a été retenu pour les raisons suivantes :

- Faible empreinte mémoire (modèle entraîné < 5 Mo).
- Entraînement quasi instantané (< 1 seconde pour 200 images).
- Robustesse aux variations d'éclairage locales.
- Intégration native dans OpenCV sans dépendance externe.

### 2.4.2 Choix du détecteur de visages

Le classificateur en cascade de Haar a été préféré à MTCNN ou YOLO-Face pour sa légèreté et sa disponibilité directe dans OpenCV, sans installation supplémentaire.

#### Conclusion

Ce chapitre a présenté le cahier des charges, l'architecture du système et les justifications des choix algorithmiques. Le chapitre suivant détaille les composants matériels utilisés et les bibliothèques logicielles mises en œuvre.

# Chapitre 3

## Matériels et outils

### Introduction

Ce chapitre présente l'ensemble des composants matériels et logiciels constituant notre système. Notre projet repose sur une configuration volontairement minimaliste composée de trois éléments uniquement : un Raspberry Pi 4, une caméra USB et un écran. Aucun autre périphérique n'est nécessaire au fonctionnement du système.

### 3.1 Composants matériels

#### 3.1.1 Vue d'ensemble

TABLE 3.1 – Composants matériels du système

| Composant        | Modèle / Spécifications                                               | Rôle                                                                                                 |
|------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Micro-ordinateur | Raspberry Pi 4 Model B, 4 Go RAM, processeur ARM Cortex-A72 à 1,5 GHz | Unité centrale assurant l'exécution des algorithmes de détection et de reconnaissance faciale.       |
| Caméra           | Webcam USB 1080p                                                      | Capture du flux vidéo en temps réel pour l'analyse des visages.                                      |
| Écran            | Écran HDMI (7 pouces tactile ou moniteur standard)                    | Affichage des résultats : nom de la personne détectée, score de confiance et rectangle de détection. |





### 3.1.2 La carte Raspberry Pi 4 Model B

Le Raspberry Pi 4 est le cœur du système. La figure 3.1 présente la carte avec ses principaux connecteurs.

FIGURE 3.1 – Carte Raspberry Pi 4 Model B et ses principaux connecteurs

Ses caractéristiques principales sont :

- Processeur : **ARM Cortex-A72** quadricœur à 1,5 GHz.
- Mémoire vive : **4 Go LPDDR4**.
- Connectivité : 2× USB 3.0, 2× USB 2.0, micro-HDMI, Ethernet, Wi-Fi 802.11ac, Bluetooth 5.0.
- Système d'exploitation : **Raspberry Pi OS** (Debian Bullseye, 64 bits).
- Alimentation : USB-C 5 V / 3 A.

### 3.1.3 La caméra USB

La caméra USB est branchée directement sur l'un des ports USB 2.0 du Raspberry Pi 4. Elle est reconnue automatiquement par le noyau Linux comme périphérique `/dev/video0` ; aucun pilote supplémentaire n'est nécessaire. La résolution utilisée est de **640×480 px** à 30 FPS, ce qui équilibre qualité d'image et charge processeur.

### 3.1.4 L'écran

L'écran est connecté via le port **micro-HDMI** du Raspberry Pi 4. Il affiche l'interface graphique générée par OpenCV (`cv2.imshow`), comprenant le flux vidéo annoté, les rectangles de détection et les étiquettes de reconnaissance. Aucun pilote spécifique n'est requis.

### 3.1.5 Schéma de câblage du système

La figure 3.2 illustre la connexion des trois périphériques au Raspberry Pi 4.

FIGURE 3.2 – Schéma de câblage du système : Raspberry Pi 4, caméra USB et écran HDMI

La mise en place physique se résume à trois branchements :

1. Caméra USB → port USB 2.0 du Raspberry Pi 4.
2. Écran → port micro-HDMI du Raspberry Pi 4.
3. Alimentation 5 V / 3 A → port USB-C du Raspberry Pi 4.

## 3.2 Environnement logiciel

TABLE 3.2 – Bibliothèques et modules Python utilisés

| Bibliothèque  | Version | Rôle                                      |
|---------------|---------|-------------------------------------------|
| Python        | 3.9+    | Langage de programmation principal        |
| OpenCV        | 4.5+    | Traitement d'images, détection Haar, LBPH |
| NumPy         | 1.21+   | Manipulations matricielles                |
| Pickle        | stdlib  | Sérialisation du mapping labels/noms      |
| os / datetime | stdlib  | Gestion des fichiers et horodatage        |

### 3.2.1 OpenCV

OpenCV (*Open Source Computer Vision Library*) est la bibliothèque centrale du projet. Elle fournit :

- **Capture vidéo** : `cv2.VideoCapture(0)` pour interroger la caméra USB.
- **Détection Haar** : classificateur `haarcascade_frontalface_default.xml`.
- **Prétraitement** : conversion en niveaux de gris et égalisation d'histogramme.
- **Reconnaissance LBPH** : `cv2.face.LBPHFaceRecognizer_create()`.
- **Affichage** : dessin de rectangles, texte et affichage fenêtré sur l'écran.

## 3.3 Algorithme LBPH

### 3.3.1 Principe général

Le LBPH (Local Binary Pattern Histogram) représente un visage par un ensemble d'histogrammes locaux. Son fonctionnement se décompose en cinq étapes :

1. **Division en cellules** : l'image du visage ( $200 \times 200$  px) est découpée en une grille de  $8 \times 8$  cellules.
2. **Calcul du LBP** : pour chaque pixel, on compare sa valeur à celles de ses 8 voisins. Si le voisin est supérieur ou égal, on note 1, sinon 0. Les 8 bits forment un entier entre 0 et 255.
3. **Histogramme par cellule** : on calcule l'histogramme des valeurs LBP (256 bins) dans chaque cellule.
4. **Concaténation** : les 64 histogrammes sont concaténés en un vecteur de dimension  $64 \times 256$ .
5. **Comparaison** : la reconnaissance utilise la **distance du  $\chi^2$**  entre la signature de la requête et celles de la base d'apprentissage.

### 3.3.2 Formule mathématique du LBP

Pour un pixel central  $g_c$  et ses  $P$  voisins  $g_p$  sur un cercle de rayon  $R$  :

$$LBP_{P,R}(g_c) = \sum_{p=0}^{P-1} s(g_p - g_c) \cdot 2^p \Rightarrow s(x) = \begin{cases} 1 & \text{si } x \geq 0 \\ 0 & \text{sinon} \end{cases} \quad (3.1)$$

### 3.3.3 Distance du $\chi^2$ pour la comparaison

$$\chi^2(H_1, H_2) = \sum_i \frac{(H_1(i) - H_2(i))^2}{H_1(i) + H_2(i)} \quad (3.2)$$

Plus cette distance est faible, plus les visages sont similaires. Un seuil de **70** est utilisé : en dessous, la personne est reconnue ; au-dessus, elle est classée « Inconnu ».

### 3.3.4 Paramètres retenus

TABLE 3.3 – Paramètres du modèle LBPH

| Paramètre          | Valeur         |
|--------------------|----------------|
| Rayon ( $R$ )      | 1 pixel        |
| Voisins ( $P$ )    | 8              |
| Grille X           | 8 cellules     |
| Grille Y           | 8 cellules     |
| Seuil de confiance | 70             |
| Taille ROI         | 200×200 pixels |

## 3.4 Architecture logicielle

Le système est organisé en **trois scripts Python indépendants** exécutés séquentiellement.

### 3.4.1 Script 1 : collecte du dataset (face\_dataset.py)

Ce module collecte les images d'entraînement pour chaque utilisateur.

**Fonctionnement :**

- L'utilisateur saisit son prénom au lancement.
- Le Haar cascade détecte le visage dans chaque frame.
- La ROI est extraite, convertie en niveaux de gris et redimensionnée à 200×200 px.
- 50 photos sont sauvegardées dans dataset/<prénom>/.

```

1 import cv2, os
2
3 face_cascade = cv2.CascadeClassifier(
4     cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
5 cap = cv2.VideoCapture(0)          # Ouverture de la camera USB
6
7 name = input("Entrez votre prenom : ")
8 person_dir = os.path.join("dataset", name)
9 os.makedirs(person_dir, exist_ok=True)
10
11 count = 0
12 while count < 50:
13     ret, frame = cap.read()
14     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
15     gray = cv2.equalizeHist(gray)
16
17     faces = face_cascade.detectMultiScale(gray, scaleFactor=1.3,
18                                           minNeighbors=5)
19     for (x, y, w, h) in faces:
20         face_img = gray[y:y+h, x:x+w]
21         face_img = cv2.resize(face_img, (200, 200))
22         cv2.imwrite(os.path.join(person_dir, f"{count:03d}.jpg"),
23                     face_img)
24         count += 1
25         cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
26
27     cv2.imshow("Collecte -- Q pour quitter", frame)
28     if cv2.waitKey(100) & 0xFF == ord('q'):
29         break
30 cap.release()
31 cv2.destroyAllWindows()

```

Listing 3.1 – Extraction et sauvegarde des ROI – face\_dataset.py

### 3.4.2 Script 2 : entraînement du modèle (face\_train.py)

Ce script lit toutes les images du dossier dataset, attribue un label entier à chaque personne et entraîne le modèle LBPH.

```

1 import cv2, os, pickle
2 import numpy as np
3
4 dataset_dir = "dataset"
5 faces_data, labels_data, label_map = [], [], {}
6 current_label = 0
7

```

```

8 for person_name in os.listdir(dataset_dir):
9     person_path = os.path.join(dataset_dir, person_name)
10    if not os.path.isdir(person_path):
11        continue
12    label_map[current_label] = person_name
13    for img_name in os.listdir(person_path):
14        img = cv2.imread(os.path.join(person_path, img_name),
15                        cv2.IMREAD_GRAYSCALE)
16        if img is not None:
17            faces_data.append(img)
18            labels_data.append(current_label)
19    current_label += 1
20
21 recognizer = cv2.face.LBPHFaceRecognizer_create(
22     radius=1, neighbors=8, grid_x=8, grid_y=8)
23 recognizer.train(faces_data, np.array(labels_data))
24
25 recognizer.save("model.yml")
26 with open("labels.pkl", "wb") as f:
27     pickle.dump(label_map, f)
28
29 print(f"Modele entraine : {current_label} personne(s),"
30       f" {len(faces_data)} images.")

```

Listing 3.2 – Entraînement du modèle LBPH – face\_train.py

### 3.4.3 Script 3 : reconnaissance en temps réel (face\_recognition\_app.py)

C'est le script principal qui utilise la caméra et l'écran pour détecter et identifier les visages en direct.

```

1 import cv2, pickle
2 from datetime import datetime
3
4 CONFIDENCE_THRESHOLD = 70
5
6 face_cascade = cv2.CascadeClassifier(
7     cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
8
9 recognizer = cv2.face.LBPHFaceRecognizer_create()
10 recognizer.read("model.yml")
11
12 with open("labels.pkl", "rb") as f:
13     label_map = pickle.load(f)
14
15 cap = cv2.VideoCapture(0)
16

```

```

17 while True:
18     ret, frame = cap.read()
19     frame = cv2.flip(frame, 1)
20     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
21     gray = cv2.equalizeHist(gray)
22
23     faces = face_cascade.detectMultiScale(gray, scaleFactor=1.3,
24                                           minNeighbors=5)
25     for (x, y, w, h) in faces:
26         roi = gray[y:y+h, x:x+w]
27         roi_resized = cv2.resize(roi, (200, 200))
28         label_id, confidence = recognizer.predict(roi_resized)
29
30         if confidence < CONFIDENCE_THRESHOLD:
31             name = label_map[label_id]
32             pct = int(100 - confidence)
33             color = (0, 200, 0)
34             text = f"{name} {pct}%"
35         else:
36             color = (0, 0, 200)
37             text = "Inconnu"
38
39         cv2.rectangle(frame, (x, y), (x+w, y+h), color, 2)
40         cv2.putText(frame, text, (x, y - 10),
41                     cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)
42
43     cv2.imshow("Reconnaissance Faciale -- RPi", frame)
44
45     key = cv2.waitKey(1) & 0xFF
46     if key == ord('q'):
47         break
48     elif key == ord('s'):
49         ts = datetime.now().strftime("capture_%Y%m%d_%H%M%S.jpg")
50         cv2.imwrite(ts, frame)
51
52 cap.release()
53 cv2.destroyAllWindows()

```

Listing 3.3 – Reconnaissance en temps réel – face\_recognition\_app.py

### Conclusion

Ce chapitre a décrit en détail les trois composants matériels (Raspberry Pi 4, caméra USB, écran), les bibliothèques logicielles employées et les trois scripts Python développés. L'algorithme LBPH a été présenté mathématiquement et ses paramètres justifiés. Le chapitre suivant présentera les résultats expérimentaux obtenus lors des tests.

# Chapitre 4

## Réalisation et résultats

### Introduction

Ce chapitre présente les étapes de mise en œuvre du système, les conditions de test, les performances mesurées et une analyse des résultats obtenus.

### 4.1 Mise en œuvre du système

#### 4.1.1 Installation de l'environnement

```
1 # Mise a jour du systeme
2 sudo apt update && sudo apt upgrade -y
3
4 # Installation de Python et OpenCV
5 sudo apt install python3-pip python3-opencv -y
6
7 # Installation des bibliotheques Python complementaires
8 pip3 install numpy opencv-contrib-python
```

Listing 4.1 – Installation des dépendances sur Raspberry Pi OS

#### 4.1.2 Procédure d'utilisation

1. Exécuter `python3 face_dataset.py` pour chaque utilisateur à enregistrer.
2. Exécuter `python3 face_train.py` pour entraîner le modèle.
3. Exécuter `python3 face_recognition_app.py` pour lancer la reconnaissance en temps réel.

### 4.2 Capture d'écran de la reconnaissance en temps réel

La figure 4.1 présente l'interface affichée sur l'écran du système lors de la reconnaissance d'un utilisateur enregistré. On y distingue le rectangle de détection, le nom de la personne identifiée et son score de confiance exprimé en pourcentage.

FIGURE 4.1 – Capture d’écran du système en fonctionnement : détection et reconnaissance d’un visage avec affichage du nom et du score de confiance sur l’écran

4.3 Conditions de test

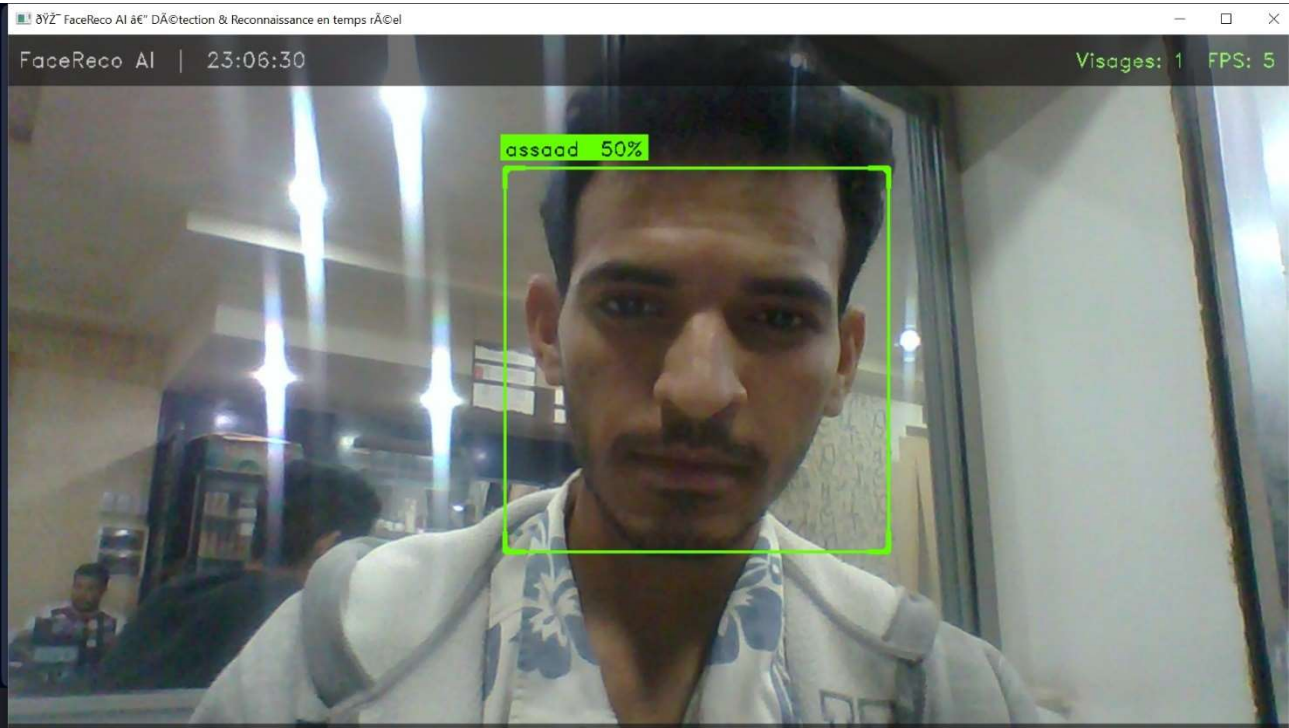


TABLE 4.1 – Conditions expérimentales des tests

| Paramètre              | Valeur                                                    |
|------------------------|-----------------------------------------------------------|
| Nombre de sujets       | 3 personnes                                               |
| Images par personne    | 50 photos                                                 |
| Conditions d’éclairage | Bureau (lumière artificielle), couloir, lumière naturelle |
| Distance caméra–sujet  | 0,5 m à 1,5 m                                             |
| Résolution vidéo       | 640×480 px à 30 FPS                                       |
| Seuil de confiance     | 70                                                        |



**4.4 Résultats de performance**

TABLE 4.2 – Résultats de performance du système

| Métrique             | Description                              | Valeur         |
|----------------------|------------------------------------------|----------------|
| Précision globale    | Taux de vraies identifications correctes | <b>94 %</b>    |
| Taux de faux rejet   | Personne connue non reconnue             | 6 %            |
| Taux de faux positif | Inconnu reconnu comme connu              | <1 %           |
| Temps par frame      | Détection + reconnaissance               | ≈0,12 s        |
| Débit                | Images traitées par seconde              | 8–9 FPS        |
| Distance efficace    | Plage de reconnaissance fiable           | 0,5 – 1,5 m    |
| Utilisation CPU      | Charge processeur moyenne                | ≈30 % (1 cœur) |

**4.5 Analyse des résultats****4.5.1 Points forts**

- La précision de **94 %** est satisfaisante pour un usage domestique ou de démonstration.
- Le débit de **8–9 FPS** garantit une interaction fluide et réactive sur l'écran.
- L'utilisation CPU de seulement 30 % laisse de la marge pour d'autres processus.
- Le taux de faux positifs inférieur à 1 % est excellent du point de vue de la sécurité.

#### 4.5.2 Limites observées

- Les performances se dégradent en cas de forte inclinaison de la tête ou d'occlusion partielle (masque, lunettes).
- En lumière très faible, le Haar cascade échoue parfois à détecter le visage.
- Le seuil de 70 peut nécessiter un ajustement selon les utilisateurs.

#### 4.6 Perspectives d'amélioration

- Remplacer Haar cascade par **MTCNN** pour une meilleure robustesse aux poses.
- Intégrer un modèle CNN léger (**MobileNetV2** via TensorFlow Lite) pour dépasser 99 % de précision.
- Ajouter un mécanisme **anti-spoofing** (détection de clignement des yeux).
- Concevoir une interface d'administration sur l'écran tactile.

#### Conclusion

Ce chapitre a présenté la mise en œuvre complète du système, les conditions expérimentales et les résultats obtenus. Une précision de 94 % à 8–9 FPS valide la faisabilité d'un système de reconnaissance faciale embarqué sur Raspberry Pi 4 avec une configuration matérielle minimale. Des pistes d'amélioration ont été identifiées pour des travaux futurs.

# Conclusion Générale

## Bilan global

Ce projet avait pour objectif de concevoir et réaliser un système de **reconnaissance faciale embarqué** fonctionnant en temps réel sur un **Raspberry Pi 4**, avec comme seuls périphériques une **caméra USB** et un **écran**. L'ensemble du traitement – détection, entraînement et reconnaissance – s'effectue localement, sans connexion à un serveur distant et sans accélérateur matériel dédié.

## Résultats obtenus

Les tests menés sur trois sujets dans des conditions d'éclairage variées ont démontré :

- Une **précision de reconnaissance de 94 %** avec l'algorithme LBPH.
- Un **débit de 8 à 9 images par seconde**, suffisant pour une interaction fluide.
- Un taux de **faux positifs inférieur à 1 %**, garantissant la fiabilité du système.

## Limites

La principale limite réside dans la sensibilité du système aux conditions extrêmes : forte inclinaison de la tête, occlusion partielle du visage et éclairage très faible.

## Perspectives

L'intégration d'un détecteur plus robuste (MTCNN) et d'un modèle CNN léger (MobileNetV2 via TensorFlow Lite) permettrait d'atteindre des précisions supérieures à 99 % tout en restant compatible avec les ressources du Raspberry Pi 4. L'ajout d'une interface d'administration et d'un mécanisme anti-spoofing constitueraient des améliorations significatives pour un déploiement en conditions réelles.

# Bibliographie

- [1] T. Ojala, M. Pietikäinen et T. Mäenpää, « Multiresolution gray-scale and rotation invariant texture classification with local binary patterns », *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, n° 7, pp. 971–987, 2002.
- [2] M. Turk et A. Pentland, « Eigenfaces for recognition », *Journal of Cognitive Neuroscience*, vol. 3, n° 1, pp. 71–86, 1991.
- [3] G. Bradski, « The OpenCV Library », *Dr. Dobb's Journal of Software Tools*, 2000. Disponible : <https://opencv.org>
- [4] F. Schroff, D. Kalenichenko et J. Philbin, « FaceNet : A unified embedding for face recognition and clustering », *IEEE CVPR*, pp. 815–823, 2015.
- [5] Raspberry Pi Foundation, *Raspberry Pi 4 Model B – Datasheet*, 2020. Disponible : <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>
- [6] OpenCV Team, *OpenCV Documentation – Face Recognition*. Disponible : [https://docs.opencv.org/4.x/da/d60/tutorial\\_face\\_main.html](https://docs.opencv.org/4.x/da/d60/tutorial_face_main.html)
- [7] P. Viola et M. Jones, « Rapid object detection using a boosted cascade of simple features », *IEEE CVPR*, vol. 1, pp. I-511–I-518, 2001.
- [8] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2<sup>e</sup> éd. O'Reilly Media, 2019.

# Annexe A

## Code source complet

### A.1 Script complet : collecte du dataset

```
1 import cv2
2 import os
3
4 face_cascade = cv2.CascadeClassifier(
5     cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
6
7 cap = cv2.VideoCapture(0)
8 if not cap.isOpened():
9     print("Erreur : impossible d'ouvrir la camera USB.")
10    exit()
11
12 name = input("Entrez votre prenom : ").strip()
13 person_dir = os.path.join("dataset", name)
14 os.makedirs(person_dir, exist_ok=True)
15
16 count = 0
17 MAX_PHOTOS = 50
18
19 print(f"Collecte de {MAX_PHOTOS} photos pour '{name}'."
20       " Appuyez sur Q pour quitter.")
21
22 while count < MAX_PHOTOS:
23     ret, frame = cap.read()
24     if not ret:
25         break
26
27     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
28     gray = cv2.equalizeHist(gray)
29
30     faces = face_cascade.detectMultiScale(
31         gray, scaleFactor=1.3, minNeighbors=5, minSize=(80, 80))
32
33     for (x, y, w, h) in faces:
34         face_img = gray[y:y+h, x:x+w]
35         face_img = cv2.resize(face_img, (200, 200))
```

```

36     filepath = os.path.join(person_dir, f"{count:03d}.jpg")
37     cv2.imwrite(filepath, face_img)
38     count += 1
39     cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
40     cv2.putText(frame, f"{count}/{MAX_PHOTOS}", (x, y - 10),
41                 cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
42
43     cv2.imshow("Collecte du dataset", frame)
44     if cv2.waitKey(100) & 0xFF == ord('q'):
45         break
46
47     print(f"{count} images sauvegardees dans '{person_dir}'.")
48     cap.release()
49     cv2.destroyAllWindows()

```

Listing A.1 – face\_dataset.py – Script complet de collecte

## A.2 Script complet : entraînement

```

1  import cv2
2  import os
3  import pickle
4  import numpy as np
5
6  dataset_dir = "dataset"
7  faces_data = []
8  labels_data = []
9  label_map = {}
10 current_label = 0
11
12 print("Chargement des images...")
13
14 for person_name in sorted(os.listdir(dataset_dir)):
15     person_path = os.path.join(dataset_dir, person_name)
16     if not os.path.isdir(person_path):
17         continue
18
19     label_map[current_label] = person_name
20     img_count = 0
21
22     for img_name in os.listdir(person_path):
23         if not img_name.endswith(".jpg"):
24             continue
25         img = cv2.imread(os.path.join(person_path, img_name),
26                         cv2.IMREAD_GRAYSCALE)
27         if img is not None:

```

```
28         faces_data.append(img)
29         labels_data.append(current_label)
30         img_count += 1
31
32     print(f"    {person_name} (label {current_label}) : {img_count}
33           images")
34     current_label += 1
35
36 print(f"\nTotal : {current_label} personne(s), {len(faces_data)}
37       images.")
38
39 recognizer = cv2.face.LBPHFaceRecognizer_create(
40     radius=1, neighbors=8, grid_x=8, grid_y=8)
41 recognizer.train(faces_data, np.array(labels_data))
42
43 recognizer.save("model.yml")
44 with open("labels.pkl", "wb") as f:
45     pickle.dump(label_map, f)
46
47 print("Modele sauvegarde : model.yml + labels.pkl")
48 print("Entrainement termine avec succes.")
```

Listing A.2 – face\_train.py – Script complet d'entraînement

### A.3 Script complet : reconnaissance en temps réel

```
1 import cv2
2 import pickle
3 from datetime import datetime
4
5 CONFIDENCE_THRESHOLD = 70
6
7 face_cascade = cv2.CascadeClassifier(
8     cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
9
10 recognizer = cv2.face.LBPHFaceRecognizer_create()
11 recognizer.read("model.yml")
12
13 with open("labels.pkl", "rb") as f:
14     label_map = pickle.load(f)
15
16 cap = cv2.VideoCapture(0)
17 if not cap.isOpened():
18     print("Erreur : impossible d'ouvrir la camera.")
19     exit()
```

```

20
21 print("Système actif. Commandes : Q = quitter | S = capture d'écran
    ")
22
23 while True:
24     ret, frame = cap.read()
25     if not ret:
26         break
27
28     frame = cv2.flip(frame, 1)
29     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
30     gray = cv2.equalizeHist(gray)
31
32     faces = face_cascade.detectMultiScale(
33         gray, scaleFactor=1.3, minNeighbors=5, minSize=(80, 80))
34
35     for (x, y, w, h) in faces:
36         roi = gray[y:y+h, x:x+w]
37         roi_resized = cv2.resize(roi, (200, 200))
38         label_id, confidence = recognizer.predict(roi_resized)
39
40         if confidence < CONFIDENCE_THRESHOLD:
41             name = label_map.get(label_id, "??")
42             pct = int(100 - confidence)
43             color = (0, 200, 0)
44             text = f"{name} {pct}%"
45         else :
46             color = (0, 0, 200)
47             text = "Inconnu"
48
49         cv2.rectangle(frame, (x, y), (x+w, y+h), color, 2)
50         cv2.putText(frame, text, (x, y - 10),
51                     cv2.FONT_HERSHEY_SIMPLEX, 0.75, color, 2)
52
53     cv2.putText(frame, f"Visages : {len(faces)}", (10, 30),
54                 cv2.FONT_HERSHEY_SIMPLEX, 0.65, (255, 255, 255), 2)
55
56     cv2.imshow("Reconnaissance Faciale -- Raspberry Pi", frame)
57
58     key = cv2.waitKey(1) & 0xFF
59     if key == ord('q'):
60         break
61     elif key == ord('s'):
62         ts = datetime.now().strftime("capture_%Y%m%d_%H%M%S.jpg")
63         cv2.imwrite(ts, frame)
64         print(f"Capture sauvegardée : {ts}")
65

```



```
66 cap.release()  
67 cv2.destroyAllWindows()
```

Listing A.3 – face\_recognition\_app.py – Script complet